

1. Prompt Strategy Document

Chain-of-Thought (CoT) Prompting Pattern Justification:

The decomposition into backend-first layers follows a dependency-driven architecture where each layer enables the next. Starting with data models (FR2, FR4) creates the foundation for structured logging - without defining what constitutes a usage log or policy rule, no subsequent functionality can be built. The logging API and middleware come next because FR4 (automatic logging) must be operational before FR5 (compliance checking) can be implemented - compliance rules can only evaluate data that already exists. The compliance engine service then builds on this logging infrastructure, directly supporting FR5's requirement for policy checking. Finally, the feedback formatting layer and UI components address FR6 (explanations) and FR7 (ethical guidance), which depend on having compliance results to display. This order ensures each component can be tested independently and prevents the common pitfall of building compliance logic without the data pipeline to support it.

Persona-Based Prompting Pattern Rationale:

The senior full-stack developer persona with GDPR and edtech specialization directly maps to our non-functional requirements. GDPR experience (NFR1) ensures the implementation considers data minimization in schema design and includes proper DELETE endpoints. Policy engine expertise (NFR5) influences how we structure configurable rules separate from core logic. Educational platform background (NFR6) shapes the feedback mechanisms to be pedagogically sound rather than purely punitive. Audit logging experience (NFR4) guides the non-blocking middleware design that maintains performance while capturing necessary data.

Contrastive Explanation Debugging Pattern Application:

This pattern addresses the complexity of policy rule evaluation where multiple rules might interact in unexpected ways. When a compliance check returns a false positive or negative, providing the expected outcome alongside the actual API response and the relevant policy rule JSON enables systematic debugging. For example, if a student's usage is incorrectly flagged for "full generation" when they only requested editing help, comparing the log data against the rule pattern reveals whether the issue is in rule configuration, log data structure, or evaluation logic. This approach proved necessary because compliance engines often fail in ways that aren't obvious from surface-level inspection.

Strategy Overview with Requirement Traceability:

The four-step implementation sequence (backend foundation -> logging middleware -> compliance engine -> frontend integration) ensures each requirement builds on working prerequisites. Step 1 establishes MongoDB schemas for UsageLog (FR2,

FR4) and PolicyRule (FR5, NFR5), with GDPR-compliant fields and deletion capabilities (NFR1). Step 2 implements auto-logging middleware (FR4) that captures data asynchronously (NFR4) and structures it according to FR3's conciseness requirement. Step 3 builds the compliance service (FR5) that evaluates logs against course-specific rules (NFR5) and generates educational feedback (FR6, NFR3, NFR6). Step 4 creates React components that consume these services while strictly separating UI from business logic, ensuring maintainability and testability.

2. Example Prompt

a. Role and System ContextYou are a senior full-stack developer specializing in GDPR-compliant educational technology platforms with extensive experience designing audit logging systems and configurable policy engines. You understand that compliance features must balance institutional requirements with student learning outcomes, and you prioritize non-blocking architectures that maintain user experience while ensuring data integrity.

b. Project ContextWe are building AIGuidebook, a platform where students document and review AI usage in coursework. You are implementing the AI Usage Logging & Compliance feature group (FR2-FR7), which forms the foundation for later analytics features. Based on the dependency analysis from Task 1.3, FR4 (automatic logging) enables FR5 (compliance checking), which in turn enables FR6 (explanatory feedback). Authentication will be integrated later, so this module must function independently with proper error handling for unauthenticated states.

c. Technical Requirements

Functional Requirements with Implementation Guidance:

FR2: Student documentation of AI tools - Implement a UsageLog schema that captures tool name, contribution description, and assignment context. The form interface should guide students through structured input while remaining flexible enough for diverse use cases.

FR3: Concise structured documentation - Design the logging interface with progressive disclosure, showing only essential fields initially with optional detailed sections. Use clear labels and examples to reduce cognitive load.

FR4: Automatic logging - Create Express middleware that intercepts AI interactions to capture tool, timestamp, assignment ID, and assistance type without blocking the main request-response cycle. Handle edge cases like missing assignment context gracefully.

FR5: Policy compliance checking - Build a compliance service that evaluates each log against course-specific PolicyRule documents. Implement both synchronous checking during logging and batch checking during submission to catch issues early.

FR6: Clear compliance explanations - Structure violation objects to include rule ID, human-readable name, explanation of what triggered the rule, and specific suggested actions. Format responses to be immediately actionable for students.

FR7: Ethical guidelines feedback - Extend the compliance service to include educational messages that precede violation details, framing feedback as learning support rather than punishment.

Non-functional Requirements with Implementation Context:

NFR1: GDPR compliance - Include user ID fields for data ownership, implement a DELETE endpoint that removes all user logs upon request, and ensure data minimization by logging only required fields. Store policy rules separately from user data.

NFR3: Detailed compliance warnings - Each violation must contain the specific rule triggered, an explanation accessible to students, and concrete corrective actions. Avoid technical jargon in user-facing messages.

NFR4: Non-blocking background operation - Implement auto-logging middleware that runs asynchronously after sending the response, using worker threads or message queues if necessary. Never delay the primary user interaction for logging or compliance checks.

NFR5: Course-specific configurable rules - Design PolicyRule schema with courseId and assignmentId fields for granular targeting. Store rules as JSON structures that can be modified without code changes, and implement rule evaluation that gracefully handles missing or malformed rules.

NFR6: Educational priority in feedback - Structure compliance responses to include an educational message field that appears before violation details. Design messages to explain why rules exist, not just what was violated.

d. Output Format and Constraints

Provide:

- Complete folder structure with file purposes
- Backend implementation:
 - MongoDB schemas with validation and GDPR considerations
 - Express routes with proper error handling
 - Controllers that delegate to services
 - Logging service with async operations
 - Compliance service with rule evaluation logic
 - Non-blocking auto-logging middleware with fallback error logging

- DELETE endpoint with verification
- Frontend implementation:
 - UsageLogForm with loading states and error boundaries
 - ComplianceFeedbackPanel that handles empty states and renders violations educationally
 - EthicalGuidanceCard that displays proactively
 - API service with retry logic and error transformation
- Example course policy rule JSON showing configuration options
- Detailed explanation mapping each implementation component to specific FR/NFR IDs, including how error cases are handled

Constraints: Use async/await with proper try-catch blocks, maintain clean separation between layers, implement modular compliance engine that can be extended with new rule types, ensure all user-facing text is in props for future internationalization, include comprehensive error handling for network failures and malformed data.

e. Expected Structured Response

```

backend/
  models/
    UsageLog.js      # Schema with user ID, timestamps, tool data, indexes for GDPR
  deletion
    PolicyRule.js    # Schema with course/assignment targeting, rule conditions,
  educational messages
  routes/
    logRoutes.js     # POST /logs, GET /logs, DELETE /logs/user/:userId (GDPR)
  controllers/
    logController.js # Request validation, response formatting, error handling
  services/
    loggingService.js # Async log creation with retry logic, bulk operations
    complianceService.js # Rule evaluation, violation generation, educational messaging
  middleware/
    autoLoggingMiddleware.js # Non-blocking log capture with error isolation
  utils/
    validators.js    # Input validation for GDPR compliance
    errorHandler.js  # Centralized error formatting

frontend/
  features/
    usage-compliance/
      components/
        UsageLogForm.jsx      # Form with validation, loading states, error display
        ComplianceFeedbackPanel.jsx # Renders violations with educational priority
        EthicalGuidanceCard.jsx # Proactive guidance display
      hooks/
        useUsageLogs.js      # Custom hook with loading/error states
        useComplianceCheck.js # Debounced compliance checking
      services/
        usageApi.js          # API client with retry logic, error transformation
  
```

types/ index.js	# Type definitions matching backend schemas
utils/ formatters.js	# Format compliance data for display

Compliance Output Example with Educational Priority:

```
{
  "status": "warning",
  "educationalMessage": "Great job using AI for editing help! Remember that AI should enhance your thinking, not replace it. Here's how to make your submission even stronger:",
  "violations": [
    {
      "ruleId": "NO_FULL_GENERATION",
      "ruleName": "Full AI-generated answers prohibited",
      "explanation": "The 'explanation' section appears to be entirely AI-generated based on style analysis. This course policy allows editing assistance but requires your original analysis.",
      "suggestedAction": "Rewrite the explanation in your own words, then use AI only to check grammar and clarity.",
      "severity": "medium"
    }
  ],
  "policyContext": {
    "courseId": "CS101",
    "ruleCount": 3,
    "lastUpdated": "2024-01-15"
  }
}
```

Implementation Mapping:

Component	Requirements Addressed	Key Design Decisions
UsageLog Schema	FR2, FR4, NFR1	Includes user ID for GDPR deletion, minimal required fields, indexes for performance
PolicyRule Schema	FR5, NFR5	JSON-based rules for configuration without code changes, course targeting
AutoLoggingMiddleware	FR4, NFR4	Runs after response sent, isolated error handling to prevent main flow disruption
ComplianceService	FR5, FR6, FR7, NFR3, NFR6	Educational messages prepended to violations, rule evaluation with fallbacks
DELETE /logs/user/:userId	NFR1	Verifies authentication (to be integrated later), cascading deletion

Component	Requirements Addressed	Key Design Decisions
UsageLogForm	FR2, FR3	Progressive disclosure, client-side validation, optimistic updates
ComplianceFeedbackPanel	FR6, NFR3	Renders educational message first, violations with actionable suggestions
EthicalGuidanceCard	FR7, NFR6	Proactive display based on usage patterns, configurable content

Error Handling Strategy:

All backend services implement try-catch blocks with specific error types. The auto-logging middleware catches and logs errors without failing the main request. The compliance service validates rule JSON before evaluation, falling back to "rule unavailable" messages for malformed rules. The frontend API service includes exponential backoff retry logic for transient failures and transforms backend errors into user-friendly messages. Each React component includes error boundaries to prevent cascade failures.